

Southwest
Tech

Python Programming Foundations

Week 4 - Debugging, Testing, & Reading Structured Code
Day 3: "Validation Testing and Justifying a Fix"

1

Southwest
Tech

A Fix Is Stronger When It Is Checked

A fix is not complete just because the error disappeared.
A believable fix has evidence.



Today, focus on:

- ▶ what changed
- ▶ what was checked
- ▶ what evidence supports the result
- ▶ how you explain the fix or modification

2

Southwest
Tech

BUG → FIX → CHECK



1. BUG

Something isn't working as expected.

Example:
Tasks don't show up in the list.

→



2. FIX

Make a change to solve the problem.

Example:
Add missing code so tasks are saved and displayed.

→



3. CHECK

Run it and see if it works now.

EVIDENCE:
Tasks now show up in the list.

A Stronger Fix

3

Southwest
Tech

Evidence, Repair, Validation

Week 4 follows a professional pattern:

- ▶ gather evidence
- ▶ repair or modify
- ▶ validate behavior
- ▶ explain the result



Validation is how you move from "I think it works" to "I checked that it works."

4

Southwest
Tech

THE DEBUGGING ARC

A loop that builds better understanding.



1. EVIDENCE

Gather facts about what is happening.

- What did I expect?
- What happened?
- Where does it differ?
- What is the data?

Goal:
Understand the problem clearly.

→



2. REPAIR

Make a small, focused change to address the root cause.

- Change one thing
- Keep it simple
- Save and run again

Goal:
Fix the cause, not just the symptom.

→



3. VALIDATION

Run and check results immediately.

- Does it work now?
- Did anything break?
- Test the important paths

Goal:
Confirm the fix with evidence.

→



4. EXPLANATION

Explain what was wrong, what changed, and why it works now.

- What was the bug?
- What did I change?
- Why does it work?

Goal:
Turn the fix into understanding.

Each time through the arc, your code gets better — and so do you.

Evidence, Repair, Validation

5

Southwest
Tech

One Run Is Not Enough

One successful run may only prove one path.

Better evidence checks more than one case:

- ▶ normal value
- ▶ boundary value
- ▶ value that should fail or choose another path
- ▶ modified behavior after a change

More checks create stronger confidence.



6

ONE RUN vs. SEVERAL SMALL CHECKS
More small checks = stronger confidence.

LEFT: ONE SUCCESSFUL RUN

You run it once. It works... for now.

! WHAT THIS TELLS YOU

- A tested case.
- You don't know about edge cases.
- Bugs can still be hiding.

Low confidence.
Easy to break without knowing.

Good habit: Try a few small cases every time you make a change. It takes minutes and saves hours.

RIGHT: SEVERAL SMALL CHECKS

You try different small cases. Each check builds confidence.

NORMAL CASE
Add a couple of tasks and show them.

BOUNDARY CASE
Show tasks when the list is empty.

ALTERNATE VALUES
Add many tasks and show them.

HIGH CONFIDENCE.
You've tested important paths. Bugs have fewer places to hide.

Stronger code!

One Run Is Not Enough

7

What Counts as Success Today

Working baseline -> compare structure -> revise one meaningful thing -> explain the improvement.

↓

A useful revision should make the program:

clearer	easier to test	easier to change	easier to explain
---------	----------------	------------------	-------------------

8

What We Will Use Today

- ▶ expected-versus-actual checks
- ▶ simple test cases
- ▶ multiple values
- ▶ optional 'assert'
- ▶ pytest recognition
- ▶ repair justification

These tools help us support claims about code behavior.

9

DEBUGGING TOOLBOX
Simple tools to find, fix, and understand bugs.

1. EXPECTED / ACTUAL CHECKS

Compare what you expected with what actually happened.

Expected: 10
Actual: 8
Not the same → keep investigating.

2. SIMPLE TEST CASES

Start with small, clear cases that are easy to reason about.

- Empty list
- One item
- Two items
- Typical case

3. MULTIPLE VALUES

Try different values to see patterns and edge behavior.

Try: 0, 1, -1, 2, 5, 10
"a", "hello", 12, 133, 3.14, 11

4. OPTIONAL ASSERT

Use assert to state what should be true at key points.

result = add(2, 3)
assert result == 5
If it fails, you know exactly where.

5. PYTEST RECOGNITION

Recognize perfect when you use it. You don't need to use it yet.

Run: `python -m pytest test.py`
Run: `python -m pytest test.py::test_add`
Run: `python -m pytest test.py::test_add::test_add_2_3`

6. REPAIR JUSTIFICATION

When you fix a bug, state why it was the right change.

Bug: Off-by-one in loop range.
Fix: Changed `range(10)` to `range(11)`.
Why: Last item was never checked, so it was missed.

GOOD DEBUGGING HABITS

- Change one thing at a time.
- Re-run and check right away.
- Keep notes on what you learn.
- Stop when you can explain the fix.

GOAL
Understand the problem, trust your fix, and explain it clearly.

Toolbox

10

What We Will "Skip" For Now

- ▶ pytest Mastery
- ▶ Mocking
- ▶ Full test suites
- ▶ Profiling
- ▶ Test-Driven Development (TDD) workflow

Today's required target is:

- ▶ show the check
- ▶ explain the repair
- ▶ justify why the result is believable

11

PARKED FOR LATER
Useful later, not today's focus.

INHERITANCE

One class can inherit attributes and methods from another class.

POLYMORPHISM

Different objects can respond to the same method call in their own way.

DECORATORS

A way to add extra behavior to functions or methods.

LARGE CLASS HIERARCHIES

Designing and managing many related classes in a bigger structure.

We'll use these ideas soon.
For now, let's build a strong foundation.

Parked For Later

12

Expected Behavior Can Be Checked

Expected behavior can be checked with simple output.

Example pattern:

```
Input: 69
Expected: False
Actual: False
Result: matches
```

The check should connect directly to the behavior you are claiming.

13

CHECK CARD

	INPUT What I gave the program.	69
	EXPECTED What I expected to get.	False
	ACTUAL What the program actually returned.	False
	RESULT How the actual compares to expected.	MATCHES Actual matches expected.

Checked Behavior

14

Demo #1

"Simple Test Cases"

15

What to Watch For

Watch one function checked with several values.

Trace:

- ▶ input value
- ▶ expected result
- ▶ actual result
- ▶ whether the result matches

Different values can reveal different problems.

16

TESTING ONE FUNCTION

Different values help reveal different outcomes.

Function being checked: `is_passing(score)`

TEST 1	TEST 2	TEST 3
Input (score) 90	Input (score) 70	Input (score) 69
Expected True	Expected True	Expected False
Actual True	Actual True	Actual False
Result: Matches	Result: Matches	Result: Matches

KEY POINT: More than one value is being checked.
This builds confidence that the function works as expected.

Demo 1: Simple Test Cases

17

Boundaries Matter

Boundary values often reveal logic mistakes.

For a passing score rule, useful checks include:

- ▶ '90' should pass
- ▶ '70' should pass
- ▶ '69' should not pass

The cutoff value is where mistakes often hide.

18



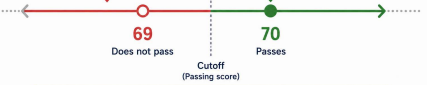
PASSING SCORE CUTOFF

Passing score is 70 or higher.

BELOW THE CUTOFF
 Does not pass
 Example: 69

ON THE CUTOFF
 Passes
 Example: 70

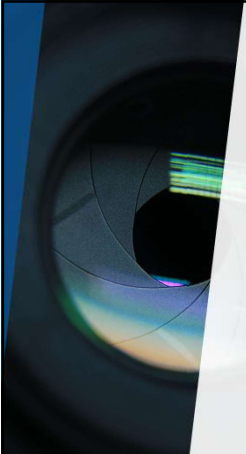
70 is the
 boundary
 value.



KEY IDEA:
 Scores less than 70 do not pass. Scores of 70 or higher pass.
 Testing both 69 and 70 helps confirm the rule.

Boundaries Matter

19



Optional Demo

“Assert” Basics”

20

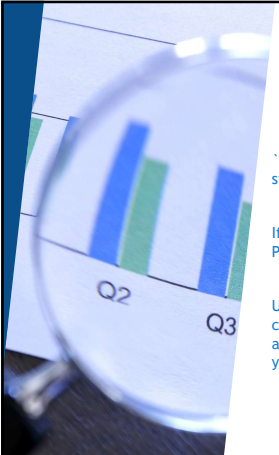


What to Watch For

'assert' checks whether a statement is true.

If the statement is false, Python raises an error.

Use it as a lightweight check (not as a new assignment burden unless your instructor requires it).



21



Testing Tool Progression

22



Testing Tool Progression

Validation can grow over time.

Beginner progression:

- ▶ expected-versus-actual print checks
- ▶ 'assert' checks
- ▶ pytest recognition
- ▶ later: full test suites and professional workflows

The idea is the same:
check behavior against an expectation.



23



Testing Tool Progression

VALIDATION GROWS OVER TIME

You don't have to climb all the rungs today.



4 FUTURE IDEAL

FULL TEST SUITES
 Build larger, organized test suites that cover more cases and keep you confident.

3 PYTEST RECOGNITION
 Recognize pytest when you see it. You don't have to use it yet.

2 ASSERT CHECKS
 Use assert to automatically check key points and catch problems early.

1 START HERE

EXPECTED-VS-ACTUAL
 Use print checks to compare what you expected with what you actually got.

expected result
 ==
 expected

expected ==
 actual: 10
 expected: 10

KEY IDEA: Validation skills build step by step. Each step makes your code more reliable. Climb at your own pace.

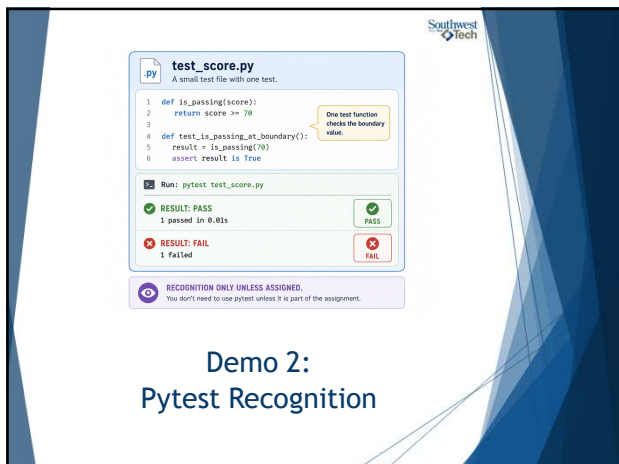
24



25



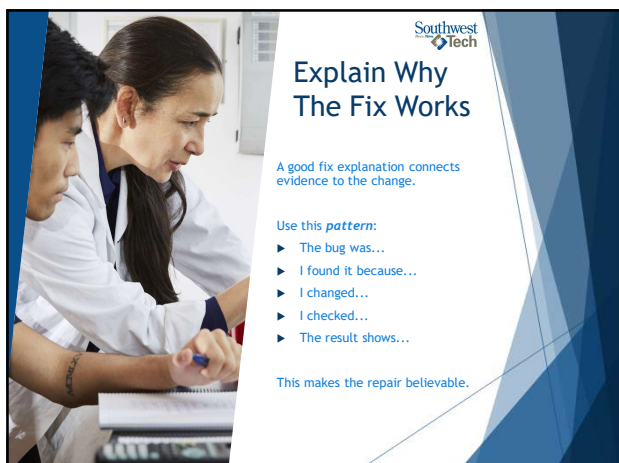
26



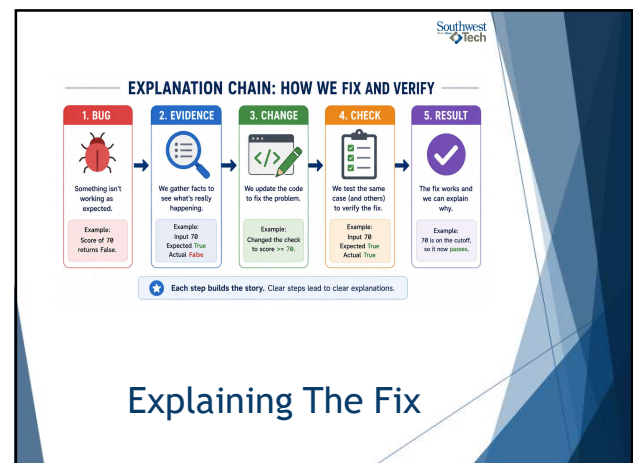
27



28



29



30

Show The Check, Explain The Repair

Evidence without explanation is incomplete.

Explanation without evidence is weak.

Strong submissions include:

- corrected or modified code
- visible check or output
- short explanation of what changed
- reason the result is acceptable

31

Finish Assignment #6

For Assignment 6, check that your submission includes:

- ▶ corrected `.py` file
- ▶ at least one debugging clue
- ▶ expected-vs-actual or test evidence
- ▶ explanation of the bug
- ▶ explanation of why the fix works

32

A6 CLOSEOUT: DEBUGGING COMPLETE
We found the issue, fixed it, and proved it works.

1. CORRECTED FILE
grade_checker.py

```
1 def is_passing(score):
2     return score >= 70 # fixed: was > 70
3
4 def test_boundary():
5     assert is_passing(70) is True
6
7 Change: > 70 → >= 70
```

2. DEBUGGING CLUE
What led us to the bug?

- Input 70 returned **False**.
- The rule says 70 should pass.

→ Clue: The code used **> 70** instead of **>= 70**.

3. CHECK OUTPUT
Print out fix works

Input	Expected	Actual	Result
69	False	False	✓ Matches
70	True	True	✓ Matches
90	True	True	✓ Matches

✓ All checks match. The fix works!

4. EXPLANATION NOTE
Why this fix is correct.

- ✓ The passing rule is 70 or higher.
- ✓ Using **>= 70** includes 70.
- ✓ Our tests now confirm the function behaves as required.

Takeaway: Test the boundary value to confirm the rule.

Finish Assignment #6

33

Finish Assignment #7

For Assignment 7, check that your submission includes:

- ▶ modified `.py` file
- ▶ class or structure explanation
- ▶ attributes and methods identified
- ▶ one working modification
- ▶ comparison to procedural or function-based code

34

A7 CLOSEOUT: CLASS MODIFICATION COMPLETE
We modified the class, understand its parts, and proved it works.

Modified Class File
task_tracker.py

```
1 class TaskTracker:
2     def __init__(self):
3         self.tasks = []
4
5     def add_task(self, task):
6         self.tasks.append(task)
7
8     def show_tasks(self):
9         if not self.tasks:
10            print("No tasks yet.")
11        else:
12            for i, t in enumerate(self.tasks, 1):
13                print(f"{i}. {t}")
```

Class Anatomy (Labels for Understanding)

- Class:** TaskTracker - The blueprint for our tracker.
- Attribute:** self.tasks - Holds the list of tasks.
- Methods:**
 - `add_task(task)`: Adds a task to the list.
 - `show_tasks()`: Displays all tasks.

Output Sample (Proof it Works)

```
Created a TaskTracker
Added 3 tasks
Displayed the tasks
```

A7 ACHIEVED!
We modified the class, explain its parts, and our change works.
Code + understanding + working output = complete.

What We Proved
The modification works and produces the expected output.

Useful Evidence Includes:

- 1. Review Lecture slides
- 2. Complete Lab
- 3. Submit assignment

Finish Assignment #7

35

Evidence For Week 4

It should show that you inspected, changed, and checked the code.

Useful evidence includes:

- ▶ corrected or modified files
- ▶ expected-vs-actual examples
- ▶ debug output or traceback clue
- ▶ simple checks or test cases
- ▶ explanation in your own words
- ▶ AI-use note if AI was permitted and used

36

EVIDENCE BOARD
Show the fix, prove it works, and explain what you learned.

1. CORRECTED / MODIFIED FILES

```
File: grade_checks.py
1 def is_passing(score):
2     return score >= 70 # fixed: was > 70
3
4 def test_boundary():
5     assert is_passing(70) is True
6
7 Change made: > 70 => >= 70
```

2. EXPECTED VS. ACTUAL EXAMPLES

Input (score)	Expected (before fix)	Actual (after fix)	Result
69	False	False	✓ Match
70	True	False	✗ Mismatch
90	True	True	✓ Match

After the fix, all results match what we expect.

3. DEBUG OUTPUT (BEFORE FIX)

Running tests before the fix:

```
> python run_tests.py
Test 69 ... PASS
Test 70 ... FAIL ← problem
Test 90 ... PASS
5 Failures, 2 passed
```

Claim: The boundary value 70 was failing
Code used: < 70 increased to >= 70

4. SIMPLE CHECKS (AFTER FIX)

Quick manual checks:

```
>>> is_passing(69)
False
>>> is_passing(70)
True
>>> is_passing(90)
True
```

Manual checks confirm the function works.

5. EXPLANATION NOTE

What was wrong?
The function used < 70, which excluded 70.

What did we change?
We changed < to >= so that 70 counts as passing.

Why this fix is correct:

- The rule says 70 or higher should pass.
- Using >= 70 includes 70.
- All tests and manual checks now match.

We fixed the bug and proved it works.

6. OPTIONAL: AS-USE NOTE

All use cases for:

- ☐ Explaining the boundary condition
- ☐ Supporting test cases
- ☐ Finding errors in the fix

Human decisions:

- ☐ Found the problem
- ☐ Changed the code
- ☐ Ran tests and verified results
- ☐ Wrote this explanation

AI helped but did the thinking, testing, and final decision.

REMEMBER: Good evidence shows the problem, the fix, and proof it works. Show your work Explain clearly Prove it works

Evidence

37

Closing Success Check

If you can prove what changed, the fix is believable.

Before submitting work, ask:

- ▶ What was wrong or unfamiliar?
- ▶ What did I change?
- ▶ What did I check?
- ▶ What evidence supports the result?
- ▶ Can I explain it in my own words?

38

Southwest Tech



Thank you!

Have Fun!

39